SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA1516

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I **R.JAIDEEP(192525441)**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: **R.JAIDEEP**

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud.
   Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems.
   Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
   Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

1.Cloud Auto-Scaling and Load Balancing for Seasonal E-Commerce Traffic

A company may experience **unpredictable traffic spikes during seasonal sales**, such as Black Friday, Diwali, Christmas, or New Year offers. During these periods, thousands or millions of customers may access the website simultaneously. If the company uses a fixed number of servers, the servers may become overloaded, causing **slow response times, application crashes, or downtime**. Cloud computing solves this problem through **auto-scaling, load balancing, and monitoring tools**.

1. Auto-Scaling

Cloud auto-scaling automatically increases or decreases the number of computing resources according to the current workload. When traffic increases, the cloud platform launches additional virtual machines or application instances. When traffic decreases, unnecessary instances are removed.

For example, during a major online sale, the number of users visiting an e-commerce website may suddenly increase from 10,000 to 100,000 users. Auto-scaling detects the increased CPU utilization, memory usage, request rate, or other predefined metrics and automatically adds more servers. After the sale ends and traffic becomes normal, the system reduces the number of servers. This provides **flexibility, better resource utilization, and cost efficiency** because the company pays for additional resources mainly when they are required.

2. Load Balancing

A **load balancer** distributes incoming customer requests across multiple servers instead of allowing all requests to reach a single server. It checks the availability and performance of servers and directs traffic to healthy instances.

For example, if an e-commerce application has five servers during a sale, the load balancer can distribute customer requests among all five servers. If one server becomes unavailable, the load balancer automatically stops sending requests to that server and redirects users to healthy servers. This prevents a single server from becoming a bottleneck and improves **availability, performance, and fault tolerance**.

3. Reducing Downtime and Improving User Experience

The combination of auto-scaling and load balancing helps prevent system overload. Auto-scaling provides additional resources when demand increases, while load balancing distributes traffic efficiently among those

resources. Therefore, users are less likely to experience **website crashes, long loading times, failed transactions, or connection errors**.

This improves the overall user experience because customers can search for products, add items to their carts, make payments, and track orders without significant interruption. It also reduces the possibility of losing customers and revenue because of website downtime.

4. Role of Monitoring Tools

Monitoring tools continuously observe the performance and health of cloud resources. They collect information such as CPU utilization, memory usage, network traffic, response time, number of requests, error rates, and server health.

When a predefined threshold is reached, monitoring systems can trigger alerts or automatically activate scaling policies. For example, if CPU utilization remains above 70% for several minutes, the monitoring system can notify the administrator or trigger auto-scaling to add more instances. Monitoring also helps identify failed servers, unusual traffic patterns, and performance problems before they significantly affect customers.

5. Real-World E-Commerce Scenario

Consider an e-commerce company such as Amazon during a major festival sale. A large number of customers may simultaneously search for products, view product pages, place orders, and make payments. The application can use a load balancer to distribute requests across multiple application servers. When traffic increases rapidly, auto-scaling can automatically add new server instances. When traffic falls after the sale, the additional instances can be removed.

Monitoring tools continuously track traffic, server utilization, application errors, and response times. If one server fails, the load balancer redirects requests to healthy servers, while auto-scaling can create replacement capacity when required.

2.Cloud Storage for Secure Backup and Disaster Recovery

Organizations generate large amounts of backup data such as databases, documents, application files, customer records, and system configurations. Storing this data securely is important because hardware failures, cyberattacks, accidental deletion, or natural disasters can cause permanent data loss. **Cloud storage** provides durability, redundancy, encryption, access control, and cost-effective storage for backup data.

1. Durability and Data Redundancy

Cloud storage platforms store data across **multiple physical devices, servers, and sometimes different data centers**. This redundancy ensures that the failure of one disk or server does not result in data loss. Cloud providers also use replication and automatic data recovery mechanisms to maintain multiple copies of important information.

For example, when an organization uploads a backup file, the cloud provider may replicate it across different storage systems. If one storage device fails, another copy can be used. Many cloud storage services also provide extremely high durability, making them suitable for long-term backup and archival requirements.

2. Encryption and Access Control

Backup data may contain sensitive information such as financial records, employee information, customer details, or business documents. **Encryption** protects this information by converting readable data into an unreadable format. Data can be encrypted both **during transmission** and **while stored in the cloud**.

Access control provides another layer of security. Organizations can define which employees, applications, or administrators are allowed to access specific backup files. **Role-Based Access Control (RBAC), identity management, multi-factor authentication, and least-privilege permissions** can prevent unauthorized users from accessing or modifying sensitive data.

3. Cost Advantages

Cloud storage can be more economical than maintaining traditional storage infrastructure. In a traditional system, an organization must purchase storage servers, hard disks, backup devices, cooling systems, electricity, and maintenance services. Additional hardware is also required when storage capacity increases.

Cloud storage follows a **pay-as-you-use model**, allowing organizations to pay mainly for the storage and services they consume. Cloud providers also offer different storage classes. Frequently accessed data can be

stored in standard storage, while old backup data can be moved to lower-cost archival storage. This reduces infrastructure and maintenance expenses and eliminates the need for large upfront hardware investments.

4. Disaster Recovery Use Case

Consider a **hospital** that stores patient records, medical reports, and application databases. If its local data center is damaged by a fire, flood, hardware failure, or ransomware attack, the hospital could lose access to important information. The organization can maintain encrypted backups in cloud storage and configure automatic backup schedules.

During a disaster, the organization can restore the required databases and applications from the cloud backups to another environment. Because backup copies are stored separately from the primary infrastructure, the organization can recover its services more quickly and reduce data loss and downtime.

Conclusion

Cloud storage provides a reliable solution for storing large volumes of backup data. **Data replication and redundancy improve durability**, while **encryption and access control protect sensitive information**. Pay-as-you-use pricing and archival storage options can reduce costs compared with maintaining traditional storage infrastructure. In disaster recovery, cloud backups allow organizations to restore critical data and applications after failures, helping ensure **business continuity, data protection, and faster recovery**.

3. Content Delivery Networks (CDNs) for Global Mobile Applications

A mobile application deployed globally must provide fast and reliable services to users located in different countries and regions. If all users connect to a single server or data center, users who are geographically far away may experience **high latency, slow loading times, buffering, and poor application performance**. A **Content Delivery Network (CDN)** solves this problem by distributing application content through multiple geographically distributed servers. CDNs are especially useful for applications involving video streaming, online gaming, software downloads, images, and other large amounts of data.

1. Role of Content Delivery Networks

A **CDN is a distributed network of servers** located in different geographical regions. These servers are commonly called **edge servers or edge locations**. Instead of every user requesting content directly from the company's main server, the CDN delivers cached content from an edge server located closer to the user.

For example, suppose a mobile application has its main data center in the United States, but users are located in India, Europe, and Australia. If every user has to communicate directly with the US data center, users in distant regions may experience greater network delays. With a CDN, copies of frequently requested content can be stored at edge locations closer to these users. Therefore, a user in India can receive content from an Indian or nearby edge location instead of communicating with the US data centre for every request.

2. How CDNs Reduce Latency

Latency is the time taken for data to travel between a user's device and the server. Physical distance, network congestion, and the number of network connections can increase latency.

A CDN reduces latency by placing cached content closer to users. When a user requests a video, image, JavaScript file, application update, or other static content, the CDN checks whether the content is already available at a nearby edge server. If it is available, the edge server delivers it directly to the user.

This reduces the distance that data needs to travel and decreases the number of network hops. As a result, users experience **faster loading times and quicker responses**.

3. Importance of Edge Servers

Edge servers are an important part of a CDN because they bring computing and content delivery closer to the end user. They can store frequently accessed content and, depending on the cloud architecture, perform certain processing tasks closer to users.

For example, a user watching a video in India can receive video segments from an edge server located closer to the user rather than downloading every segment from a server located in another continent. Similarly, a gamer can connect to a nearby edge location to reduce network delay.

Edge servers improve the user experience by providing:

- Faster content delivery.
- Lower network latency.
- Reduced buffering.
- Faster application loading.
- Better performance during traffic spikes.
- Reduced load on the origin server.

4. CDN Caching

Caching is one of the main techniques used by CDNs. When a user requests a particular piece of content, the CDN may store a copy of that content at the edge server. Subsequent users requesting the same content can receive it from the cached copy.

For example, if thousands of users download the same promotional video during a product launch, the CDN does not need to retrieve the video from the origin server for every user. Instead, the video can be served from nearby edge locations.

This reduces the workload on the main application servers and allows them to concentrate on dynamic operations such as **login, payment processing, user profiles, game state, and database transactions**.

5. Global Availability Through Cloud Providers

Cloud providers support global application deployment by providing **multiple regions and availability zones** around the world. An organization can deploy application components in different geographical locations and use a CDN to deliver content from nearby edge locations.

If one region experiences a major failure, traffic can be redirected to another healthy region. Cloud providers can also provide services such as **global load balancing, DNS-based traffic management, health monitoring, automatic scaling, and cross-region replication**.

This architecture improves application availability because the application does not depend entirely on one physical data center. Users can continue accessing the application even when a particular region experiences technical problems.

6. Real-Time Streaming Example

Consider a global video-streaming mobile application such as Netflix. Millions of users may watch movies, television programs, or live events from different countries at the same time.

If every video request were sent to a single central server, the server could become overloaded and users in distant regions could experience buffering. A CDN can distribute video content across many edge locations. When a user starts watching a video, the required video segments can be delivered from an edge location close to that user.

For example, users in India can receive content from nearby CDN infrastructure, while users in Europe can receive content from European edge locations. This reduces latency and network congestion and provides smoother playback.

For **live streaming**, the architecture can be combined with adaptive bitrate streaming. The application can provide different video qualities depending on the user's network conditions. If the network becomes slower, a lower-quality stream can be delivered to prevent continuous buffering.

7. Gaming Application Example

CDNs are also valuable for online gaming applications. A global multiplayer game may have players located in India, Europe, North America, and other regions. Players need quick responses because delays can negatively affect gameplay.

Cloud providers can deploy game servers in multiple regions and direct players to an appropriate nearby region. Edge locations can also deliver game updates, maps, images, videos, and other static resources quickly.

For example, when a new game update is released, millions of players may download the same large update file. A CDN can distribute the update through edge servers, reducing the load on the central infrastructure and allowing players to download it faster.

4. Cloud Compliance and Data Security for Financial Institutions

Financial institutions handle highly sensitive information such as **bank account details, credit card information, customer identities, payment records, and financial transactions**. Therefore, when a bank or financial company uses cloud services, it must ensure that its data remains secure and complies with applicable laws and regulations. Cloud providers support this through compliance certifications, auditing tools, logging, monitoring, encryption, identity management, and regional data-storage controls.

1. Regulatory Compliance and Auditing

Cloud providers offer infrastructure and security features that help organizations meet regulatory requirements. Depending on the country and type of financial service, organizations may need to follow regulations and standards related to **data protection, financial reporting, privacy, security, and auditability**.

Cloud providers commonly maintain compliance certifications and provide documentation describing their security controls. They also offer tools that allow financial institutions to configure their own security policies and generate audit evidence.

2. Role of Logging

Logging records important activities occurring within cloud systems. Logs can include information about user logins, file access, database operations, configuration changes, API calls, and security events.

For example, if an employee accesses a customer's financial transaction record, the system can record **who accessed the data, when it was accessed, what resource was accessed, and what operation was performed**. These records help organizations investigate suspicious activities and provide evidence during security audits.

Logs should be protected from unauthorized modification and retained according to the organization's regulatory and business requirements.

3. Role of Monitoring

Monitoring continuously observes cloud infrastructure and applications for unusual or potentially dangerous activities. Security monitoring can identify repeated failed login attempts, unusual data transfers, unexpected changes in configurations, or abnormal access patterns.

For example, if a customer's account is normally accessed from India but suddenly receives hundreds of login attempts from an unusual location, monitoring systems can generate an alert. Security teams can investigate the event and take appropriate action.

Monitoring therefore helps organizations move from simply recording security events to **detecting and responding to potential threats proactively**.

4. Role of Encryption

Encryption protects sensitive financial information by converting readable data into an encoded form that cannot be easily understood without the appropriate decryption key.

Financial institutions should use encryption for both:

- **Data at rest:** Information stored in databases, cloud storage, and backup systems.
- **Data in transit:** Information transferred between users, applications, databases, and cloud services.

For example, when a customer performs an online banking transaction, encryption helps protect the transaction information while it travels between the customer's mobile application and the financial institution's cloud infrastructure.

Proper **key management** is also important because unauthorized access to encryption keys could compromise protected information.

5. Data Sovereignty

Data sovereignty means that data is subject to the laws and regulations of the country or jurisdiction where it is stored or processed. A financial institution may therefore be required to keep certain customer or transaction data within a specific country.

Cloud providers support this requirement by offering multiple **geographical regions and data-center locations**. The company can select an appropriate region and configure its applications and storage services so that regulated information remains within the required jurisdiction.

The organization should also verify where backups, replicas, logs, and disaster-recovery copies are stored. Simply selecting a local primary region may not be sufficient if backup data is automatically replicated to another country.

5. Containerization, Virtual Machines, and Kubernetes

Containerization is a modern approach to application deployment in which an application and its required libraries, dependencies, and configuration files are packaged into a **container**. Containers make applications easier to develop, test, deploy, and scale consistently across different environments. They are widely used in cloud computing and microservices architectures.

1. Containers vs. Virtual Machines

The main difference between containers and virtual machines (VMs) is how they use the underlying operating system.

A **virtual machine** includes a complete guest operating system along with the application and its dependencies. A hypervisor manages multiple VMs on the same physical server. Because every VM requires its own operating system, VMs generally consume more memory, storage, and CPU resources.

A **container**, on the other hand, shares the host operating system's kernel while keeping applications isolated from one another. A container contains only the application and the libraries or dependencies it needs. Therefore, containers are generally **smaller and faster to start** than VMs.

Feature	Containers	Virtual Machines
Operating system	Share host OS kernel	Each VM has its own guest OS
Resource usage	Low	Higher
Startup time	Usually seconds or less	Usually longer
Performance overhead	Low	Higher
Density	Many containers can run on one host	Fewer VMs generally fit on the same resources
Portability	Highly portable	Portable but usually heavier

Because containers require fewer resources, organizations can run a larger number of application instances on the same infrastructure. This improves resource utilization and can reduce infrastructure costs.

2. Benefits of Containerization

Containers package an application together with its dependencies. This helps ensure that the application behaves consistently across development, testing, and production environments.

For example, a developer can create an application container on a laptop and deploy the same container image to a cloud environment. The application does not need to be completely reconfigured for every environment.

Containers also support **rapid deployment, isolation, easy scaling, and efficient resource utilization**. If a new version of an application is required, developers can create a new container image and deploy it without modifying the underlying infrastructure extensively.

3. Role of Kubernetes Orchestration

When an organization has only a few containers, they can be managed manually. However, a large application may contain hundreds or thousands of containers. Managing these containers individually becomes difficult.

Kubernetes is a container orchestration platform that automates the deployment, management, networking, and scaling of containers. It allows developers and administrators to define the desired state of an application, while Kubernetes works to maintain that state.

Kubernetes can automatically:

- Deploy containers across available servers.
- Restart failed containers.
- Distribute workloads across nodes.
- Scale application instances based on demand.
- Perform service discovery and load balancing.
- Support rolling updates with minimal interruption.
- Replace unhealthy application instances.

4. Deployment and Scaling

Suppose an application normally requires three container instances. During a sudden increase in users, the application may require ten instances. Kubernetes can automatically increase the number of running containers according to predefined scaling rules.

When traffic decreases, unnecessary instances can be reduced. This provides **elasticity** and prevents organizations from continuously running more resources than required.

Kubernetes also improves availability. If one container fails, Kubernetes can automatically restart it or create a replacement container. If a server becomes unavailable, workloads can be moved or recreated on other available nodes.

5. Portability Across Cloud Environments

One of the major advantages of containers is **portability**. A container image can run on a developer's computer, a private data center, or a public cloud platform as long as the required container runtime and supporting environment are available.

For example, an organization may develop an application using containers and later deploy it on different cloud environments such as Amazon Web Services, Microsoft Azure, or Google Cloud. Kubernetes can provide a common orchestration approach across these environments.

This reduces **vendor lock-in** and makes it easier for organizations to move workloads between cloud providers or use a hybrid-cloud strategy.

6. Microservices-Based Application Example

Consider an **online shopping application** developed using a microservices architecture. Instead of building the entire application as one large program, it can be divided into independent services such as:

User Service → Product Service → Order Service → Payment Service → Notification Service

Each service can run inside its own container. For example, the payment service may have five container instances while the product service may have ten instances because it receives more requests.

Kubernetes can manage these containers, distribute traffic between instances, restart failed containers, and automatically scale individual services when demand changes.

During a major shopping event, the **Product Service and Order Service** may receive a large increase in traffic. Kubernetes can increase their container instances without unnecessarily scaling services such as Notification Service. This makes the application more efficient and responsive.